

A01712379
Laura Cintora Cendejas
Campus Querétaro
28 de Mayo del 2025
Después de clase | Act-Integradora-4
Grafos y algoritmos
TC1031.850

En esta evidencia se implementó un archivo llamado ALDWGraph.h en donde se usó grafos para este tipo de datos que tienen diferentes conexiones distintas de IP, asimismo se entienden como nodos que están conectados por medio de aristas. Por lo tanto, el grafo muestra la red de comunicación en donde cada nodo está creando una conexión, ya sea uno o más.

Se presenta por medio de una lista que es adyacencia, permitiendo hacer una modelación de la forma en que se conecta, pero se realiza de manera eficiente porque en algunos casos se vuelve más pesado y no se realiza de forma correcta. Esto haciendo una lista de adyacencia mejor debido al consumo de la memoria a comparación de otras estructuras.

Cabe mencionar que el uso de la estructura de grafo, se aplicó algoritmos avanzados, uno de ellos es el de Dijkstra, el cual encuentra las rutas de forma óptima y puede detectar específicamente sus patrones de comportamiento. Esto nos dice lo importante que es para un análisis de redes o incluso para la ciberseguridad debido a su precisión en la detención.

También se utilizó el Binary Heap para poder identificar las IPs que tienen un alto grado de salida, esto sirve para ordenar y acomodar los registros por medio de la fecha. Otra de sus ventajas es la eficiencia con la inserción y extracción de su valor mínimo o máximo porque se realiza en un tiempo de $O(\log n)$.

El heap también nos ayuda a tener los elementos de forma ordenada sin que se requiera el uso de sort, esto permite ahorrar tiempo y recursos. En nuestro proyecto se mantuvo una estructura de forma eficaz para poder hacer la extracción de las 5 IPs, determinando de forma rápida el primer IP que se comporta como bot master con base a la fecha.

La implementación que se realizó, fue que el grafo se ejecutó con una lista de adyacencia, esto nos permitió recorrer todos los nodos y sus conexiones con la complejidad de $O(V + E)$ en otras palabras V es el número de nodos y E es el número de aristas, entonces se ejecuta cuando itera las conexiones y calcula los grados de salida para poder encontrar todos los vecinos.

En el Binary Heap, se usó varias complejidades, una de ellas es la inserción, la extracción de máximo y mínimo, el peek () en donde consulta el primero con complejidad $O(1)$ y por último el heap_sort que está sobre un vector de $O(n \log n)$. Esto ayudando a que el sistema pueda ejecutarse bien al momento de tener un volumen superior de datos. Ya

que es fundamental que el sistema pueda llevar acciones como almacenamiento, lectura y análisis de datos de red en un tiempo real debido a una buena estatura de lógica.

Aquí explicaré de mi justificación del uso del algoritmo de Dijkstra que fue elegido por ser el más apropiado para los grafos, asimismo cuando se ejecuta es con una fila de prioridades, esto hace que sea más óptimo porque son grados altos y dispersos casi como los que se modelan en redes de computadores además su complejidad es de $O((V + E) \log V)$. Permitiendo determinar de manera eficiente todas las rutas más cortas que empieza desde el bot master hacia las IPs, identificando cuáles se requieren para un mayor esfuerzo para llegar al objetivo.

A continuación se muestra el pseudocódigo:

Inicio

- paths_bot_master_ordered: es el vector de objetos de Dijkstra_path_info el cual están ordenados de mayor a menor distancia

Proceso:

1. Primero se obtiene el primer camino del vector que es el de mayor distancia,

`camino_mayor_esfuerzo ← paths_bot_master_ordered[0]`
2. Se realiza la extracción del nodo destino, en donde le cuesta más hacer su alcance o su objetivo.

`ip_dificil ← camino_mayor_esfuerzo.to`
3. Hace la extracción de la ruta que se tiene el nodo

`ruta ← camino_mayor_esfuerzo.path`
4. Imprime el IP de origen (bot master)
5. Para cada paso en ruta va a imprimir la dirección del IP que está en el nodo actual

Este es el proceso después de que se aplicó el algoritmo de Dijkstra que es desde la IP que se detectó como un bot master, entonces determinada de todas las IPs del grado cuál está más lejos de la red y además considerar la red.

A continuación se muestra el código como se visualiza en el lenguaje de programación de C++ el cual se encuentra en el archivo de main.cpp

```
auto paths_bot_master = registros_graph.dijkstra(bot_master); //  
Se calculan las rutas del bot master
```

```

    Heap<ALDWGraph<Registro>::Dijkstra_path_info, std::less<>>
    heap_paths(paths_bot_master); // Se crea un heap con los paths del
    bot master
    auto paths_bot_master_ordered = heap_paths.heap_sort(); // Se
    ordenan los paths del bot master por distancia

    std::ofstream distancia_out("distancia_botmaster.txt"); // Se
    crea un archivo para guardar las distancias
    for (auto& path : paths_bot_master_ordered) // Se imprimen las
    distancias
    {
        distancia_out << "IP: " << path->to->get_ip_str() << "
    Distancia: " << path->distance << endl; // Se imprime la distancia
    }
    distancia_out.close(); // Se cierra el archivo

    Registro ip_harder = *(paths_bot_master_ordered[0]->to); // Se
    guarda el ip de mayor esfuerzo desde Bot Master
    cout << "IP de mayor esfuerzo desde Bot Master: " <<
    ip_harder.get_ip_str() << endl; // Se imprime el ip de mayor
    esfuerzo desde Bot Master

    std::ofstream bot_master_attack("ataque_botmaster.txt"); // Se
    crea un archivo para guardar el ataque del bot master
    auto attack_path = paths_bot_master_ordered[0]->path; // Se
    guarda la ruta del bot master
    bot_master_attack <<
    paths_bot_master_ordered[0]->from->get_ip_str() << endl; // Se
    imprime el ip del bot master
    for (auto& registro : attack_path) // Se imprimen los ips del
    ataque
    {
        bot_master_attack << registro.first->get_ip_str() << endl;
    // Se imprime el ip del registro
    }
    bot_master_attack.close(); // Se cierra el archivo

```

Asimismo recorre el vector path para imprimir los pasos se determinaba con una complejidad de $O(k)$ en donde la k es uno de los números de nodos que se encuentran en el camino, por lo tanto, su complejidad del algoritmo con base en a la impresión es un poco más complicada $O(k)$ pero es eficiente y es el correcto para este proyecto.

Conclusión:

Este reto fue uno de los que batallé, primero por el tema de los grafos y más por una solución que ahora es más estable y eficiente al usar los grafos al igual que el heap entonces hace que se puedan implementar algoritmos complejos como el Dijkstra y se logra identificar varios patrones que son importantes por medio de su comportamiento dentro de

la red. Cada día se vuelve más complejo el código por las diferentes implementaciones de algoritmos que se aplican, pero este código se ha adaptado bastante bien a comparación de otros entornos y esto me ha ayudado a reforzar mis conocimientos.

Referencias Bibliográficas:

1. Adjacency list. (s/f). Programiz.com. Recuperado el 29 de mayo de 2025, de <https://www.programiz.com/dsa/graph-adjacency-list>
2. Agrawal, S. (2016, marzo 21). Dijkstra's Shortest Path Algorithm using priority_queue of STL. GeeksforGeeks. https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-using-priority_queue-stl/
3. Cppreference.com. (s/f). Cppreference.com. Recuperado el 29 de mayo de 2025, de <https://en.cppreference.com/>
4. Google C++ style guide. (s/f). Github.io. Recuperado el 29 de mayo de 2025, de <https://google.github.io/styleguide/cppguide.html>
5. Graph data structure. (s/f). Programiz.com. Recuperado el 29 de mayo de 2025, de <https://www.programiz.com/dsa/graph>